

Test 5

test5

 Quick Submit Quick Submit Madan Mohan Malaviya University of Technology

Document Details

Submission ID

trn:oid:::1:3541525193

Submission Date

Apr 18, 2026, 9:06 AM GMT+5:30

Download Date

Apr 18, 2026, 9:10 AM GMT+5:30

File Name

major_report.pdf

File Size

2.9 MB

52 Pages

8,191 Words

44,522 Characters

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **46 Not Cited or Quoted 6%**
Matches with neither in-text citation nor quotation marks
-  **1 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 5%  Internet sources
- 2%  Publications
- 3%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 46 Not Cited or Quoted 6%**
Matches with neither in-text citation nor quotation marks
- 1 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 5% Internet sources
- 2% Publications
- 3% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	www.coursehero.com	<1%
2	Student papers	Islington College,Nepal	<1%
3	Internet	generativeaiandhci.github.io	<1%
4	Student papers	Manipal University	<1%
5	Internet	www.tfzr.rs	<1%
6	Student papers	Babes-Bolyai University	<1%
7	Internet	ijrpr.com	<1%
8	Internet	www.ijprems.com	<1%
9	Internet	www.imensosoftware.com	<1%
10	Internet	studygroom.com	<1%

11	Student papers	Canterbury Institute of Management	<1%
12	Student papers	Grimsby College, South Humberside	<1%
13	Student papers	University of Mauritius	<1%
14	Publication	Kotian, Ashish Tharanath. "Sacramento State Universe Explore the Campus Digita..."	<1%
15	Student papers	University of Wales Institute, Cardiff	<1%
16	Internet	www.theseus.fi	<1%
17	Student papers	University of College Cork	<1%
18	Student papers	Vignana Bharathi Institute of Technology	<1%
19	Internet	elib.uni-stuttgart.de	<1%
20	Internet	123dok.com	<1%
21	Internet	eprints.soton.ac.uk	<1%
22	Internet	intelleecollege.com	<1%
23	Internet	publikationsserver.thm.de	<1%
24	Internet	www.ijsred.com	<1%

25	Internet	www.nuevoweb.com	<1%
26	Internet	mdbailey.ece.illinois.edu	<1%
27	Internet	portfolio.ecs.soton.ac.uk	<1%
28	Internet	shaktitanwar.com	<1%
29	Internet	www.powwowcasino.com	<1%
30	Publication	Arvind Dagur, Karan Singh, Pawan Singh Mehra, Dharendra Kumar Shukla. "Intelli...	<1%
31	Publication	S. R. Reeja, Bore Gowda, Y. S. Rammohan, Ganesan Prabu Sankar, G. Jayalatha. "E...	<1%
32	Internet	dspace.daffodilvarsity.edu.bd:8080	<1%
33	Internet	ir.msu.ac.zw:8080	<1%
34	Internet	medium.com	<1%
35	Internet	open-innovation-projects.org	<1%
36	Internet	pdfs.semanticscholar.org	<1%
37	Internet	techlabari.com	<1%
38	Internet	www.jetir.org	<1%

39

Publication

"Proceedings of the 6th International Conference on Data Science, Machine Lear... <1%

40

Publication

Paolo Ferro, Harinadh Vemanaboina, Chander Prakash. "Computational Techniqu... <1%

**A PROJECT REPORT
ON
CAMPUS BUY**

Submitted for partial fulfilment of the Requirements for the Degree of
MASTER OF COMPUTER APPLICATION

Session (2025-26)

Submitted by

RASHI GUPTA

(University Roll No: 2024073045)

PRIYANSHU YADAV

(University Roll No: 2024073039)

YOGESH KUMAR PANDEY

(University Roll No: 2024073074)

Under the Supervision of

Supervisor

DR. SNEHA MISHRA

(ASSISTANT PROFESSOR, ITCA)



Department of Information Technology & Computer Application

Madan Mohan Malaviya University of Technology,

Gorakhpur Uttar Pradesh 273010, INDIA

(APRIL 2026)

CERTIFICATE

This is to certify that the project report titled "**CampusBuy**" submitted by **Rashi Gupta (University Roll No:2024073045), Priyanshu Yadav (University Roll No:2024073039), Yogesh Kumar Pandey (University Roll No:2024073074)** to Madan Mohan Malaviya University of Technology in partial fulfilment of the requirements for the award of the degree of Masters of Computer Application is a Bonafide record of the work carried out by them under my supervision.

Supervisor

Dr. Sneha Mishra
Assistant Professor,
Department of Information Technology & Computer Application
MMMUT, GORAKHPUR

Date:

CANDIDATE'S DECLARATION

We here declare that the report "**CampusBuy**" submitted to **Madan Mohan Malaviya University of Technology Gorakhpur** is entirely our own work. We certify that to the best of my belief and knowledge, it has no material that has been published or written by someone else previously, or any material that has been accepted for the award of any other diploma or degree from this institution or any other higher learning institution, except where acknowledgement has been made in the work.

Sign.:

Student Name: RASHI GUPTA
(University Roll No: 2024073045)

Sign.:

Student Name: PRIYANSHU YADAV
(University Roll No: 2024073039)

Sign.:

Student Name: YOGESH KUMAR PANDEY
(University Roll No: 2024073074)

Date:

APPROVAL SHEET

This project report entitled "**CampusBuy**" submitted by Rashi Gupta (2024073045), Priyanshu Yadav (2024073039) and Yogesh Kumar Pandey (2024073074) is approved for the degree of Master of Computer Applications, ITCA, MMMUT, GORAKHPUR, 273010.

External Examiner

Supervisor

Dr. Sneha Mishra

Assistant Professor,

Department of Information Technology & Computer Application

MMMUT, GORAKHPUR

Head of Department

Prof. D. S. Singh

Professor & HOD,

Department of Information Technology & Computer Application

MMMUT, GORAKHPUR

Date: _____

Place: _____

ACKNOWLEDGEMENT

It is indeed a great pleasure to express our sincere thanks to our supervisor **Dr. Sneha Mishra, Assistant Professor**, Department of Information Technology & Computer Application, Madan Mohan Malaviya University of Technology, Gorakhpur for her valuable advice and pragmatic views of the project. Her keen interest, sincere advice, and kind help throughout during the completion of this work had been a regular source of encouragement. We are thankful to **Prof. D. S. Singh**, Head of Department of Information Technology & Computer Application for providing us department labs and terminals for our Project and other teachers of Department of Information Technology & Computer Application for their friendly treatment and cooperation in the valuable advice and suggestion.

We are indebted to our parents who raised us to be self-assured and well-balanced person and always had faith in our capability to complete the things we initiated. We are also thankful to Almighty who provided us with the strength and health for the completion of the work.

Sign.:

RASHI GUPTA

(Roll No: 2024073045)

Sign.:

PRIYANSHU YADAV

(Roll No: 2024073039)

Sign.:

YOGESH KUMAR PANDEY

(Roll No: 2024073074)

LIST OF ABBREVIATIONS

API - Application Programming Interface

CSS – Cascading Style Sheets

DB - Database

DFD – Data Flow Diagram

ERD – Entity Relationship Diagram

HTML – Hyper Text Markup Language

HTTP – Hyper Text Transfer Protocol

HTTPS – Hyper Text Transfer Protocol Secure

JS – JavaScript

MERN – MongoDB, Express.js, React.js, Node.js

NoSQL – Not Only Structured Query Language

OTP – One-Time Password

REST – Representational State Transfer

UI – User Interface

UX – User Experience

URL – Uniform Resource Locator

JSON – JavaScript Object Notation

CDN – Content Delivery Network

LIST OF FIGURES

	Page No.
1. Figure 1: Level 0 DFD	13
2. Figure 2: Level 1 DFD	14
3. Figure 3: ERD Structure	15
4. Figure 4: Registration Page	33
5. Figure 5: Login Page	33
6. Figure 6: Update Profile Page	34
7. Figure 7: List Product Page	34
8. Figure 8: Home Page	35
9. Figure 9: My Products Page	35
10. Figure 10: All Products Page	36
11. Figure 11: Wishlist Page	36
12. Figure 12: Feedback Page	37
13. Figure 13: Universal Chat Page	37
14. Figure 14: Admin Dashboard Page	38

ABSTRACT

CampusBuy is a platform specific to the campus community to provide a safe way of buying, selling, and trading second-hand products amongst the students of Madan Mohan Malaviya University of Technology (MMMUT), Gorakhpur. Using the MERN stack (MongoDB, Express.js, React.js, Node.js) to create the website, the platform enables trading of academic and personal products like textbooks, electronic gadgets, laboratory tools, and other furniture.

The security and authentication of the users are ensured via institutional email-based one-time password verification, which restricts the access to genuine MMMUT registered users only. Some of the features incorporated in CampusBuy are listing of products with Cloudinary supported images, real-time global chats using Socket.IO technology, filtered searches, personalized dashboards, JWT and bcrypt password hashing for authentication purposes, and efficient storing of information using MongoDB.

CampusBuy helps in bridging the communication gap between the buyer and the seller in an efficient and user-friendly manner. Not only does it help to save money, but it also helps in saving the planet and reducing pollution by promoting the use of used materials in the campus community.

TABLE OF CONTENT

Topic	Page No.
Certificate	I
Candidate Declaration	II
Approval Sheet	III
Acknowledgement	IV
List of Abbreviations	V
List of Figures	VI
Abstract	VII
Table of Content	VIII

Chapter	Page No.
1. Introduction	1
1.1 CampusBuy Overview	1
1.2 Project Objectives	1
1.3 Problem Statement	2
1.4 Goals of the system	2
1.5 Project Scope	3
2. Literature Review	4
2.1 Existing Systems	4
2.2 Limitations of Existing System	4
2.3 Necessity of a Campus Specific Market Place	5
2.4 Comparative study	6
3. System Analysis	7
3.1 Feasibility Study	7
3.1.1 Technical Feasibility	7
3.1.2 Economical Feasibility	8
3.1.3 Operational Feasibility	8
3.2 Requirement Analysis	9
3.2.1 Functional Requirements	9
3.2.2 Non-Functional Requirements	10
3.3 Roles and Permissions of Users	10
4. System Design	12

4.1 System Architecture	12
4.2 Data Flow Diagram	12
4.2.1 DFD Level 0	13
4.2.2 DFD Level 1	13
4.3 Entity Relationship Diagram	14
4.4 Database Design	16
4.5 UI/UX Design Principles	18
4.6 Workflow of System	20
5. Technologies and Tools	22
5.1 Frontend Technologies	22
5.2 Backend Technologies	23
5.3 Database	24
5.4 Authentication	25
5.5 Real-time Communication	25
5.6 Cloud Services	25
5.7 Development Tools	26
6. Project Description	27
6.1 User Registration & Authentication	27
6.2 Product Listing Module	27
6.3 Search and Filtering of Product	27
6.4 Wishlist Functionality	28
6.5 Real Time Chat System	28
6.6 Complaint/Report Module	28
6.7 Admin Dashboard	28
6.8 Security Features	29
7. Testing And Validation	30
7.1 Testing Methodologies	30
7.2 Unit Testing	30
7.3 Integration Testing	31
7.4 System Testing	31
7.5 Test Cases and Results	31
7.6 Bug Tracking and Fixes	32
8. Output Screens	33
9. Conclusion and Future Work	39
9.1 Conclusion	39

9.2 Future Work	39
10. References	41

CHAPTER 1

INTRODUCTION

1.1 CampusBuy Overview

CampusBuy is a web-based marketplace application that is developed for the use of the university students. The application facilitates secure and convenient trading of items within a campus. It aims to be a platform where only the students of a specific university can participate, assuring them with the genuineness and trustworthiness of their dealings. Unlike a normal e-commerce market, CampusBuy is a closed loop.

The system uses the MERN stack: MongoDB, Express.js, Node.js, and React.js. This has created an up to date and responsive interface, allowing the system to be usable and accessible for users.

Students are able to post items that range from books, electronics, and accessories to everyday essentials. Users are given features such as a real-time chat for buyers and sellers, a "wish list" option, complaint/reporting options as well as a Lost and Found section.

The intent of the system is to limit the dependence on external sites, decrease any likelihood of fraud occurring, and encouraged a sense of "re-using" culture on campus.

1.2 Project Objectives

The main objectives of the CampusBuy project is to create a secure and user-friendly marketplace where students can buy and sell items on their own campus. The reason is that existing online marketplaces lack the necessary trust to allow students to freely buy and sell products among each other. Students tend to encounter non relevant ads and have issues arranging and picking up the purchased goods from students on campus.

Another main objective is to design a system where only the people in a university community can make their exchange in a regulated and authenticated system. When restricting entry to users with authentication only, then only real members of a university community can carry out the transactions with other users from the same community.

Lastly, the project also has the objective to gain practical experience about full-stack web development by providing a system with the latest and relevant features such as authentic system, instant messaging, cloud storage.

1.3 Problem Statement

Many students experience difficulties when buying or selling items on campus. Typical approaches of word-of-mouth and notice boards are not very efficient. Online marketplaces may be used to carry out these transactions. However, existing online marketplaces may not always be customized for transactions that occur on campus. The issue of frauds can rise, as these marketplaces are not restricted for students only.

Key problems include:

- Lack of a unique system to facilitate buying and selling transactions on campus.
- Difficulty in confirming the legitimacy of buyers and sellers.
- Communication challenges among students.
- Lack of a platform to report complaints or grievances.
- No way of centralizing lost and found on campus.

1.4 Goals of the system

The overall goal is to build a trusted, user-friendly, and efficient campus platform to buying and selling services/items within institution. The main goals are:

The top-most goal is to create a trusted and safe online trading place within the institution where only authenticated institution members can buy and sell products. This is to achieve by implementing institution authentication like using OTP on log in where a member must have the institutional email address to be authenticated.

A secondary goal is to make searching and listing of items more convenient. For sellers, they are able to upload item details, price and images, while for buyers, they can make advanced searching for desired item.

Efficiency in communication has also been a goal when designing the system. By providing real time chat facility the buyers and sellers can have a direct conversation through which negotiation can take place. Buyers can even ask for the details related to the product or seller to make it easier for them to reach a decision. The users security and their accountability can also be ensured by adding complaint/reporting facility. With a system to report suspicious ads or misconduct by anyone, it is easy to manage. There is a "Lost & Found" section too, in order to fulfill campus specific needs.

Apart from the stated features, it also strives to achieve an intuitive and reactive User interface, so that any user from the campus could readily make use of the interface despite any degree of technical skill they possess. Good UI helps in improving the interaction of user with the system.

Another primary objective is the sustainability and economical feasibility by motivating to transfer unwanted used products among members of the campus, which also helps to reduce wastage and cost-effective approach for students.

Technically, the system is designed to show practical use of modern web technologies such as the MERN stack with features such as real-time communication and cloud based storing solutions.

It can be extended to a large scalable system that could integrate new features such as online payment services, mobile application interface, and intelligent recommendation system at a later stage.

1.5 Project Scope

The project scope is on building the marketplace within CampusBuy where transactions are secure and only available to students within an institution. Users are authenticated and will have features to allow users to buy and sell goods, real time chat among students, manage and view listings. All of this is restricted and controlled within the community.

The following modules have been identified and designed for the CampusBuy system; Authentication of user, browsing and listing of products, searching and filtering of products, real time chatting, wish-list of products and dealing with reported products, lost and found section, and also the admin side of the platform. However, the platform doesn't include online payment, and a delivery system, mobile application access is also not a concern in the current system development and it will be done later, since it can be expanded from this system development.

CHAPTER 2

LITERATURE REVIEW

2.1 Existing Systems

Online services, particularly those concerned with peer-to-peer (C2C) transactions, have risen rapidly in the past few years. Several C2C online systems are popular, notably OLX and Facebook Marketplace, allowing customers to list, search and sell items either locally or from a distant locale.

OLX is a well-known international classified advertising website which is available on various electronic devices such as mobiles, tablets, and PCs. It allows users to sell a number of items ranging from electronics and furniture to vehicles and even services and is largely free to use in terms of listing an advertisement, direct messaging of buyers and sellers and is home to millions of active users and ads, having revolutionized the way of reading traditional ads.

Likewise, Facebook Marketplace is a feature embedded in the Facebook application used to carry out buying and selling operations within communities. The function utilizes the social network in Facebook, with chats done through Messenger, and searches done via location and interests. The website is readily available, and has become a popular spot for ordinary people to conduct business.

2.2 Limitations of Existing Systems

While these platforms, OLX and Facebook Marketplace, are highly prevalent, their drawbacks are noticeable when viewing them from the perspective of an environment like that of a college campus.

A primary drawback is the absence of any verification or trust system. Since nearly anyone can create an account and make postings on these sites, the likelihood of fake and scam listings is amplified. Buyers or users are often required to individually check the authenticity of items and people, which is not always reliable.

The next area that presents a problem is the lack of reliable payment and security systems. Transactions take place offline and in an informal environment making it more prone to conflict and lack of accountability.

They are characterized by quality control and listing variations are weak. As they hardly has any moderation on products, quality, correctness and dependability of product details is fluctuating and unsatisfying users' expectations.

Furthermore, the lack of domain-specificity of these platforms does not enable them to address a specific community but a worldwide or a region-based one, hence listing inappropriate products and an unable to identify correct product to the user's locale.

Some of the weaknesses in other areas are also:

- Inefficiency with search and filtering at certain circumstances
- Lack of system for dealing with a specific complaints or disputes
- Lack of functions such as lost and found system
- No independent system to deal with, depending on other communications and responses

All these weaknesses prove that it should have its own independent system which is more specific.

2.3 Necessity of A Campus Specific Market Place

These inadequacies in current mechanisms emphasize the necessity of an campus specific marketplace. The reasons for a need of such market place in campus is outlined here:

The user group in a campus is defined and there is a high frequency of circulation of second-hand items like books, electronic gadgets, fashion accessories etc. Also, the sense of community and trust in such user base is usually higher than the general public. However, the general websites and services failed to exploit these merits.

This lack of a targeted platform is exactly where a dedicated campus marketplace such as CampusBuy comes in. It offers:

- limited membership that is exclusively for users associated with the university community.
- Increased sense of safety, trust and ease, as all users are fellow students.
- Localized and topic specific listings, which results in fewer irrelevant clutter and easier search experience.
- In built communication system so as users can easily connect within the platform without needing other communication tool.
- Other campus specific tools such as campus lost and found, etc.

In addition, an item exchange like this is good for environment, and it is also cost efficient as users get to resell items they no longer needed to other people within campus, so there is lesser spending and wastage.

Therefore, it is very beneficial and crucial to implement such a campus-oriented online transaction place.

2.4. Comparative study

The comparative analysis between the systems existing and that to be developed CampusBuy illustrates how the latter will be able to tackle issues that a system localized for the campus faces.

Feature	OLX	Facebook Marketplace	CampusBuy
User Base	Global users	Global social network users	Campus-specific users
User Verification	Limited	Strong (Institutional authentication)	Strong (Institutional authentication)
Trust Level	Moderate	Moderate	High
Product Relevance	Low (mixed listings)	Medium	High (campus-focused)
Communication	Chat system	Messenger integration	Real-time chat (Socket.IO)
Payment System	Mostly offline	Mostly external	Planned secure integration
Complaint System	Limited	Limited	Dedicated reporting system
Lost & Found Feature	Not available	Not available	Available
Security	Basic	Basic	Enhanced

CHAPTER 3

SYSTEM ANALYSIS

This is a key step in the development of any software system, to understand the problem domain, analyze the viability of proposed solution, and specify the system requirement. System analysis is necessary in CampusBuy to make the application efficient and usable to the campus users.

3.1 Feasibility Study

Feasibility study is an indispensable part of systems analysis that assesses the practicality and feasibility of a proposed system before its design and implementation. It provides an assessment on whether a system is feasible to be developed and implemented with the current resources, constraints, and environment. For CampusBuy system, three key issues need to be considered.

3.1.1 Technical Feasibility

Technical feasibility examines if required technology, tools, or technical knowledge exists to implement a proposed system.

The CampusBuy platform is built on the MERN stack technology where M represents MongoDB, E represents Express.js, R represents React.js, and N represents Node.js. Such technologies are quite modern in use today; this can ensure performance and scalability to the proposed system. It is also easier for the two layers, front and back end, to interact with each other as they are compatible with each other and can run on many platforms. In addition to the above, other technologies were used like, Socket.IO which is mainly used for communication, and Cloudinary used for storing and managing images on cloud servers.

In terms of hardware requirements, there are no complex or advanced infrastructure required in developing and deploying the system. An ordinary computer system with internet is needed to develop, test and deploy the system. The system can be hosted on any cloud services, which is both scalable and reliable without high infrastructure investment.

In addition to the hardware requirements, a vast number of documentations, community support and readily available online help on most technologies will also ease the development and troubleshooting process. Moreover, the developer is proficient enough to develop the system based on these technologies.

Hence, technical feasibility of the CampusBuy system is confirmed as it's supported by various tools, technologies and development team expertise.

3.1.2 Economical feasibility

It measures the cost effectiveness of a project through comparing predicted benefit versus the cost for development, implementation and maintenance.

Economically CampusBuy system is feasible because it can be developed at a very low cost. Most of the technologies and frameworks used are open-source, so it can save money on licenses for expensive software. Most of the tools for development like code editor, version control system can be obtained free. Infrastructure costs is saved too by using cheap/free-tier cloud hosting service and maintenance cost is small due to the use of module system and community supports.

The advantages of the system outweigh the costs associated with building it. CampusBuy can eliminate dependence on third party services, save time of users, provide a safe platform to conduct their transactions, and improve overall campus buying and selling. It helps improve efficiency in campus transactions and transactions at campus.

The system can be upgraded in the future with more advanced features like payment gateway integration, or other value-added services that might enhance the system.

Hence, the cost involved in building the system is low, with high benefits.

3.1.3 Operational Feasibility

Operational feasibility concerns with the usability and effectiveness of the system in its operational environment and the user acceptance of the system.

The CampusBuy is suitable in terms of the context that it is targeted to the environment and users of the campuses. The students often sell or purchase second-hand products like books, accessories, electronic gadgets etc, and this system would be a solution for those transactions.

The system would be able to serve through the easily understandable interface where navigation of any task like searching for a product, listing an item, or communicating with the user is straightforward. Technical knowledge would not be a requirement for any user to work with the system.

The live chatting facility for the users will create an ease for the users in communication and it would lead to better user interaction. The system would be implemented with the facility of reporting a complaint, system security features and a proper control for admin.

The admin would be able to monitor each and every system activity and it would help in the smooth functioning of the system. The user will accept the system since their working pattern will be similar to other web-based platforms, which they are accustomed with.

In general, the CampusBuy system is practical and user friendly.

3.2 Requirement Analysis

Requirement analysis deals with finding and writing down the functional as well as non functional requirements of a system. This leads to a product, which satisfied user expectations and works at different circumstances.

3.2.1 Functional Requirements

Functional requirements describe the functionalities the CampusBuy system must offer:

The system shall include the authentication system where the users can log in safely after verifying their identity to use the service. OTP login with university credential information gives an extra security and saves unauthorized users from logging in into the system.

Users shall be able to add, modify and remove products. A product details includes product name, description, price, category and images.

The system shall provide a full search and filtering mechanism where users can search for products based on categories, keywords, etc. It will enhance user experience and save them time in searching.

The platform shall support a real-time chat function between buyers and sellers where communication for negotiation, clarification, and decision-making can happen on the platform.

Users should be able to "bookmark" items that are not immediately purchase-able but is of interest in the form of a wish-list.

The system shall support a mechanism for complaints and reporting of issues such as offensive listings or fraud activities so that they can be addressed quickly to uphold the integrity of the system.

A Lost and Found Module should be developed to help the system be as useful to the users, such as for reporting and finding items.

An administrator interface should be available to allow administrators manage the system, such as the users of the system, the listings, the complaints etc.

3.2.2 Non-Functional Requirements

Non-Functional Requirements (NFRs) are characteristics and qualities related to how the system performs. They are essentially non-functional characteristics of the system.

- High performance - the system must perform quickly, respond promptly to user input, and be able to handle many simultaneous users concurrently.
- High security - the system must take into consideration security due to its use for communication between users and storage of user data. All sensitive data must be appropriately secured. Authentication, authorization and data protection have to be implemented appropriately.
- High usability - it is required to ensure that users are able to efficiently operate with and use the system due to intuitive user interface and simplified navigation.
- Scalability - it is expected that the system would require to scale well, that is that the number of users and volume of data within the system could grow well in the future without having major consequences.
- High reliability - the system is required to perform consistently, without system failure/Errors etc.
- High maintainability - it is important that the system has been built in such way that it may be easily modified and maintained.

3.3 Roles and Permissions of Users

To allow controlled access and easy management of the system, there are few distinct roles in the system, which are discussed below.

1. Regular User (Student):

A regular user in the CampusBuy system would be an authenticated student of the institution who can access all the basic features of the platform. The role would consist of the following permissions:

- Ability to register, login to the system using institutional login/SSO.

- Ability to add, update, or remove their product listing.
- Ability to browse and search products on the market place.
- Ability to view the details of any particular product listing.
- Ability to chat with other users in real time.
- Ability to add items to wish list.
- Ability to report complaints, abuse or invalid listing.
- Ability to use lost and found.

2. Administrator (Admin):

The role of administrator would consists of some higher-level permission to maintain and manage the system:

- Ability to view and suspend/remove users and their account.
- Ability to view product listings and remove any invalid or offensive listings from the marketplace.
- Ability to view and resolve complaints.
- Ability to check system usage, activities, performance etc.
- Ability to view, delete or disable comments from users.

CHAPTER 4

SYSTEM DESIGN

The system design phase of software development is the process of translating system requirements into a plan for the architecture and design of the system. This phase determines the system components and their interactions with each other, the flow of data throughout the system, and the manner in which users interact with the system. The design of CampusBuy emphasizes user interaction and efficiency.

4.1 System Architecture

The CampusBuy application adopts a three-tier architecture built using the MERN stack (MongoDB, Express.js, React.js and Node.js). This type of architecture divides the application into three layers, allowing for modularity and scalability of the system. The three tiers are:

The frontend is built with React.js where users would directly interact with the system through the user interface. This layer will be responsive to users' inputs and render dynamic data displayed through different components, also handles state of the application and communicates with the backend via the use of API's. The React components will be reusable and contribute to the development speed.

The backend layer is built with Node.js with the help of Express.js framework. This layer will consist of business logic of the application along with the functions to interact with the database. Communication between the frontend and backend will be done through the use of RESTful APIs.

The database layer is built with MongoDB which is a NoSQL database well suited for data that is dynamic like product listings, user information, and chat conversations. The database will store data in a flexible and scalable manner using the JSON-like document structure.

Furthermore the system has also incorporated Socket.IO for real-time communication (e.g. Chatting feature) between users and Cloudinary for efficient storage of product images.

4.2 Data Flow Diagram (DFD)

A data flow diagram (DFD) depicts data entering and leaving a system and how data moves within the system and interacting with each other. The system processes, data stores, and external entities will be depicted in this diagram.

4.2.1 DFD Level 0

Level 0 DFD shows the entire system as a single process, and illustrates its interaction with external entities.

In the CampusBuy system, external entities are the User (Student) and Administrator. The process within is the system itself and it accepts information from user and produces proper response/ output accordingly.

- Inputs from the user: registration info, listing info, search query and chat messages.
- Outputs produced by the system: product info, search result, and chat responses.
- Interactions between administrator and system: administrator uses system to view operations, users and complaints.

It depicts the external view of the system and do not illustrate any internal workings of the system.

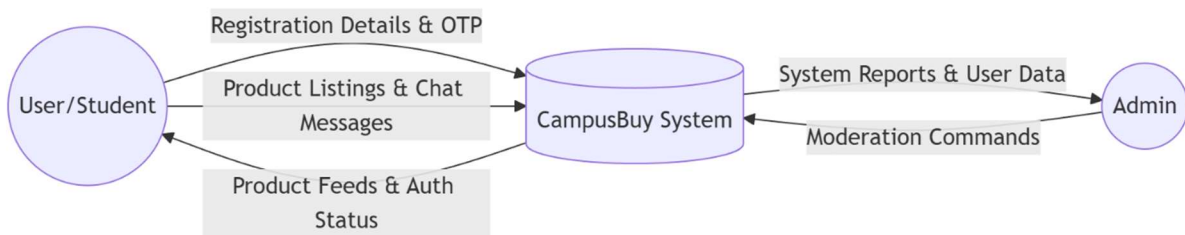


Fig 1. DFD level 0

4.2.2 DFD Level 1

This DFD details the main system by identifying the different sub-processes involved. These sub-processes help understand the flow of data inside the main system. Processes involved in CampusBuy include:

- User Authentication process: Handles login/registration of the user with OTP validation.
- Product management process: Involves adding products, modifying/updating and removing products. It also includes retrieval of the same.
- Search & Filtering process: Handles all queries sent by users and gives relevant products back to the user.

- Chat system process: Allows for instant chatting between two users using the Socket.IO implementation.
- Complaint process: The system takes care of complaints sent by the users and also sends the same to the administrator.
- Lost and Found process: Handles the lost and found notification.

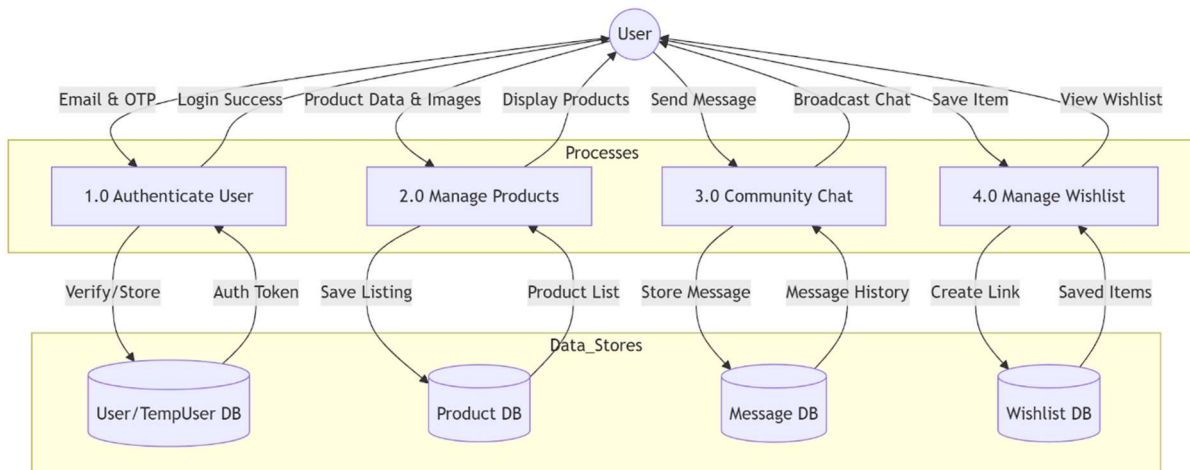


Fig 2. DFD Level 1

4.3 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram shows the logical design of the database based on entities and relationships.

Key entities in the CampusBuy system are:

- User (used for storing user data such as name, e-mail, login details, profile).
- Product (used for storing product data such as title, description, price, category, images).
- Message (used for storing chat messages between users).
- Complaint (used for storing complaints filed by the users).
- LostAndFound (used for storing lost or found item data).

Entities and relationships:

- A user can send/receive multiple messages.
- A user can file multiple complaints.
- A user can sell multiple products.
- Each product is owned by a single user.

ERD helps to structure database properly for better managing of data.

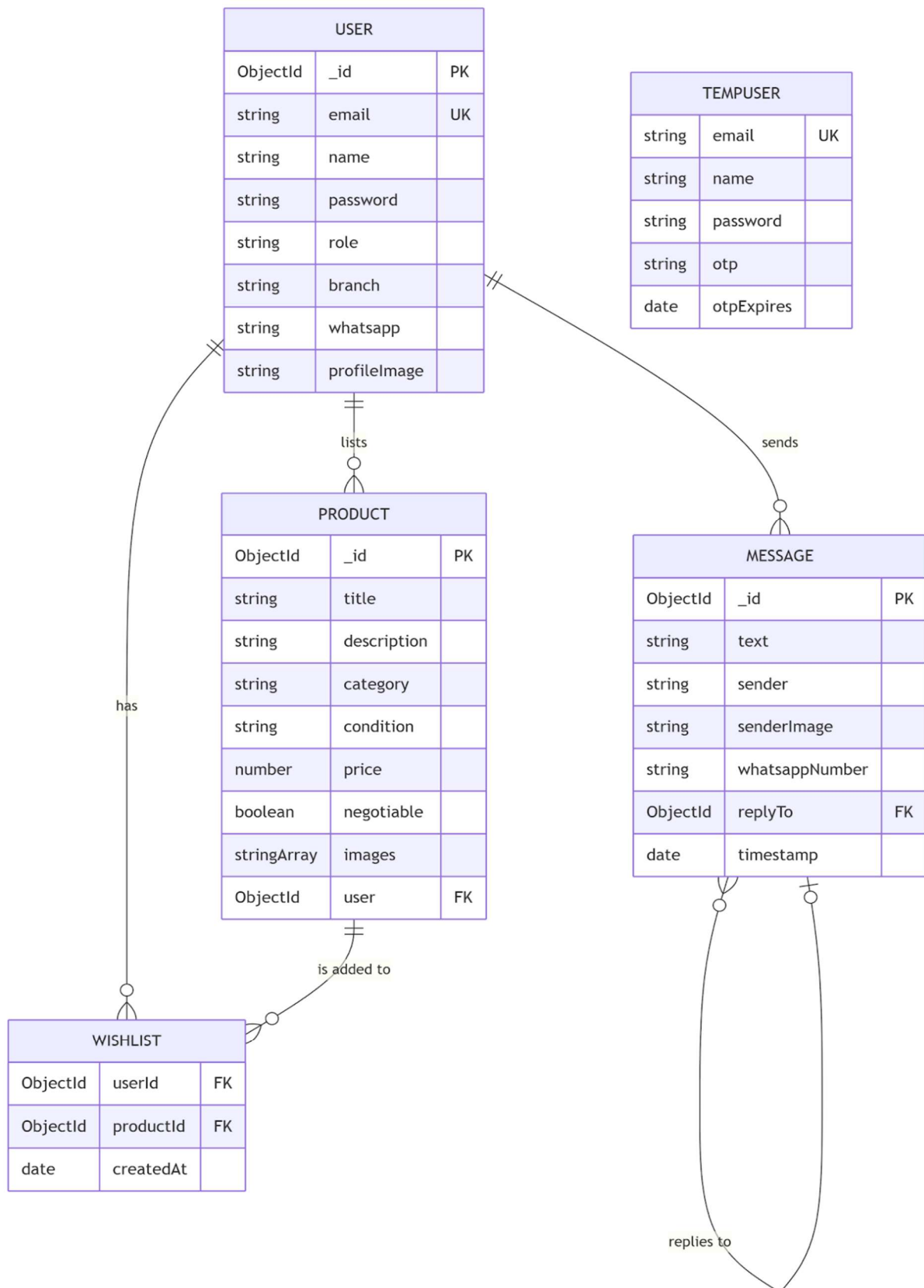


Fig 3. ERD Structure

4.4 Database Design

The database design in the CampusBuy system is central to efficient storage, retrieval and management of data. The system uses MongoDB, which is a NoSQL document database. This means that it stores data in a flexible, document based format (BSON that resembles JSON). A system that uses data in a varied and sometimes un-structured form would benefit from such flexibility and would be able to handle variations in data formats.

Database Tables/Collections:

The data in the database has been broken down into various "collections" according to their purpose. They are detailed below.

1. Users

This collection is to be used to store details about each user using the application. It should contain the following data fields, which will be stored as BSON documents.

- User ID (unique ID to distinguish users from one another)
- Name and email of user
- Encrypted password/details for OTP validation
- Profile information (for example: contact number - fields will be optional).
- User Role (Student / Admin)

2. Products Collection

This collection contains the information about all the products listed on the platform. A product document contains information such as-

- Product ID
- Product Title and Description
- Product Price and category
- Product Image URLs (stored in Cloudinary)
- Seller ID (reference to user)
- Timestamp (date of product listing)

This collection is one of the most crucial in the system. It allows you to efficiently filter and query for all the products in the system.

3. Collection of messages.

This collection will maintain a real-time chat between users. Each document in this collection will have:

- Message ID
- Sender ID and Receiver ID
- Content of the message
- Time stamp of the message
- Status (seen or unseen)

This collection can maintain a smooth chat between users.

4. Complaints Collection

In this collection, user-submitted issues/complaints are stored. A complaint will have the following fields:

- Complaint ID
- User ID of reporter
- description of the problem
- reference of product/user
- current status (pending/resolved)

The purpose is to make someone responsible of an issue and to not let the integrity of the system corrupt over time.

Database Design Considerations

Key principles applied to database design include:

- Data Normalization and Referencing- Duplication of data should be avoided, and rather references to the data (e.g. User Id within product documents) should be made in order to keep data synchronized.

- Scalability-MongoDB supports horizontal scaling, allowing the system to manage growing amounts of data.
- Performance Optimization- Indexes are utilized on commonly queried attributes like category and user ID to accelerate query speeds.
- Flexibility-The schema less property of MongoDB offers flexibility in modifying the schema on a run time basis, especially with new feature introduction.

To summarize, the database design is laid out such that it offers a stable yet efficient system in handling the data within the CampusBuy application.

4.5 UI/UX Design Principles

UI/UX design on CampusBuy aims to create a smooth, intuitive and exciting experience for the user. As CampusBuy targets students of various technical skills, usability is one of the main concern.

1. Simplicity and clarity

The design itself is to have a simple and minimalist UI where necessary functions like browse, search, and chat are within easy access for users. Labeling and navigation structures are designed to allow easy system learning.

2. Consistency

The use of colors, font, button style and layouts are same all across pages. Users will not need to learn new navigation when they shift from page to page on the application.

3. Responsive

The platform can also be used on different devices (desktop, tablet, mobile phones) and is compatible with all of them (responsive) enabling users to use it anywhere at any time.

4. Easy Navigation

Navigation is designed to be logical, clearly segmented through menus and categories. Users can seamlessly find what they are looking for through products, saved items, message inbox, and manage listings with little difficulty.

5. User Feedback and interaction

The system returns a prompt response to every user action. Some examples include;

- Messages that confirm the success of a login or product upload.
- Error messages to alert users about invalid input.
- Notifications to notify users about new messages or events.

This helps to make users more confident in and comfortable to interact with the system.

6. Accessibility

The system has been designed so that it is accessible for users with only a basic knowledge of technology, including the use of easily readable fonts, and clear color contrasts, and intuitive layout.

7. Visual Hierarchy

Key information like the product image, pricing and 'buy' buttons are accentuated to stand out to users. Ample white space, along with correct spacing and alignment enhance readability and visual appeal.

8. Design for Performance

A well-thought-out design makes the application fast, and fluid transitions prevent jank and optimize loading times, particularly for users on poor connections, where image loading or rendering could be an issue.

And overall with these principles of UI/UX design the application CampusBuy is a fun application and easy to use.

4.6 Workflow of system

System workflow describes the flow of operations and interaction of a user and the system in the CampusBuy system. The workflow diagram depicts how each module operates and interact to accomplish the functionality.

Step1: New User Registration and Authentication

User registers to the system by use of institutional mail. OTP based authentication is used for the authenticity of the user. Once registered user can access the system.

Step 2: Login

If the user is registered then the system will ask him to provide the login information. After the successful login the user will redirected to dashboard screen. From where all the functionality is available to the user.

Step 3: Browsing/searching the products

Users can browse the various available products or they can search for items using the search/filtering bar. Users can further specify searches by categories and key words.

Step 4: Product Listing

Sellers are able to add new product listing entries by imputing the required information; title, description, price, category and images, then this is stored and displayed to other users.

Step 5: Product Interaction

When looking at a product the buyer would see all relevant information such as the images of it and the description of the item. If it sparks an interest they may be able to contact the seller using the chat functionality.

Step 6: Instant messaging

Buyers and sellers are able to instantly message each other within the system to chat about product information, arrange deals, negotiate prices, etc.

Step 7: Wishlist

Users may add product to their wishlist if they want to buy that later. By this way, users could know that which product they interested.

Step 8: Complain and Report

If users face problems they can complain through the complaint system and admin will be responsible for reviewing the complaints and act accordingly.

Step 9: Admin Monitoring and Controls

Administrators control everything—they deal with users, with complaints, and keep the system stable.

The workflow guarantees all the significant system functions are processed smoothly and logically. It improves user interaction, keeps system performance and coordinates different system modules properly.

CHAPTER 5

TECHNOLOGIES AND TOOLS

CampusBuy system developed using the latest web technologies and tools available which will ensure that the system is capable of scalable, perform and reliable. This technology has been chosen based on its capability, flexibility and it is also industry widely accepted technology.

5.1 Frontend Technologies

The frontend part of CampusBuy system allows the interaction of users with the application and provides responsive and interactive interface. It helps in smooth user interface experience and effective interaction with backend services. The technologies that used in building the frontend are: React.js, HTML, CSS and JavaScript.

1. React.js

Used as the primary library for building the user interface, React.js is a JavaScript based, component based library that helps web developers write reusable UI components, making code easier to maintain and scalable. The use of the Virtual DOM ensures React.js's efficient rendering speeds by only updating and re-rendering components rather than re-loading the whole page. This increases the responsiveness of user interactions. In CampusBuy, elements like product listings, the user dashboard, the chat system, and form inputs have all been created using React.js.

2. HTML

HTML, (Hyper Text Markup Language), is used to give structure to the web pages being created. It's essential in the application to create the skeleton of the interface, including elements such as headings, forms, buttons, images, and the navigation bars. The implementation of semantic HTML will ensure accessibility is of a good standard and benefit SEO.

3. CSS

CSS (Cascading Style Sheets) is used to give the application its look and feel. It takes care of how the layout is, colors, font, spacing, responsiveness, etc. In CampusBuy, CSS is used to create a clean and consistent UI throughout the application. To make it work on all screen sizes

(desktop, tablets, mobile) responsive design features such as media queries and flexible layouts are used.

4. JavaScript

JavaScript is used to create the interactivity of the frontend. It manages user interactions like submitting forms, clicking on buttons, navigation, etc. Also, it is responsible for sending API requests to the backend, getting response from the backend, and updating the UI according to the response. Such as form validation, dynamic UI updates and states.

React's state and props system also allow for a fast and simple way to manage the data within the application so that the user interface keeps up to date with the data in the application. This will be important for features such as real-time updates, product filtering and chat.

In conclusion, the above technologies React.js, HTML, CSS, and JavaScript work together to offer a performance efficient, and fast responding front-end framework for the CampusBuy application. The user's experience with the application will be smooth and attractive.

5.2 Backend Technologies

The backend is the heart of the CampusBuy system where most of the business logic and the servers-side procedures will be carried out. It deals with data manipulation and security of data flow between the front end and database, and responsible for implementing features such as user authentication, management of products and messaging system and admin control panel.

1. Node.js

Node.js is a server-side runtime environment that allows us to execute javascript not only in a browser but also outside in a server. Node.js uses the V8 JavaScript engine from chrome and is known for high performance due to its non-blocking event-driven I/O. The non-blocking I/O allow it to handle multiple clients at the same time with almost no lag making it a suitable choice for a real-time application like CampusBuy.

Another benefit that has led to the adoption of Node.js is its handling of asynchronous events. Asynchronous events such as a database query, an external API request or file read-write operations allow Node.js to prevent other operations from becoming blocking and keep the system fast when many users hit the system. In the CampusBuy platform, the server-side logic

including the user request handling, authentication logic and database access has been done using Node.js.

2. Express.js

Express.js is lightweight and flexible web application framework built on top of Node.js. This is used in CampusBuy to take care of the server side operations and make developing APIs easier. Express.js is efficient for managing requests, responses, and application logic, enabling quicker and more structured backend development.

Highlights of Express.js:

- Routing: Routes different HTTP requests like GET, POST, PUT, DELETE etc.
- Middleware support: used for authentication, validation, and error handling.
- API development: useful for developing RESTful APIs for frontend and backend interaction.
- Integrations: compatible with Node.js and MongoDB.

The overall use of Express.js in the system is to provide efficiency, structure and scalability to the backend.

5.3 Database (MongoDB)

CampusBuy application use MongoDB to store and manage application data. MongoDB stores data in a document type, it is kind of a JSON formatted data structure. It stores and processes data in JSON-like, which enables various kind of data structures be stored efficiently.

MongoDB is schema less and flexible that traditional relation database requires fixed schema. Developers can update data structure quickly while application developed. The product details which users upload and the comments posted are good examples for dynamic data that MongoDB can handle well.

Besides schema less, MongoDB has performance, scalability, efficient querying. It supports index for faster query and can be scaled horizontally for bigger storage.

5.4 Authentication

The CampusBuy authentication mechanism helps in the security aspect by restricting access to only authenticate users.

This mechanism works by utilizing the use of OTP (One-Time Password). Once an institution's employee or student logs on the platform, a unique OTP will be send to their institutional e-mail address or mobile number.

This system replaces the traditional user authentication by password because this kind of mechanism helps in reducing password-related vulnerabilities, this also makes the login process more manageable, as a temporary OTP would suffice without needing the user to remember a long-kept password.

The institution authentication guarantees the safety of students from any campus, preventing unwanted entries within that institutional domain.

5.5 Real-time Communication (Socket.IO)

Socket.IO is implemented in the CampusBuy system in order to facilitate real-time two-way communication among users.

Socket.IO is responsible for real-time instant messaging between buyers and sellers which provides speed and efficiency in the system. Unlike the traditional communication over HTTP which is based on the request-response nature, Socket.IO will maintain an persistent connection among client and server.

Socket.IO can be applied in the following features:

- Real-time Chatting between users
- Real-time message sending to users
- Real-time page update without refresh.

5.6 Cloud Services (Cloudinary for Image Storage)

CampusBuy relies on Cloudinary for all image storage and management as it is a cloud based media management service.

The image hosting service uses Cloudinary to store and manage all images of products within the application. This provides a robust and scalable system to handle image uploads, storage

and delivery. This frees up some load from the application server, with images hosted on a cloud based server.

The advantages gained by using Cloudinary were:

- Effective storage and retrieval of images
- Optimization and compression of images (automatic)
- Allows access to images via URLs
- Efficient serving of images from a CDN

5.7 Development Tools (VS Code, Git, Postman)

The CampusBuy system uses a number of tools for development in order to boost the efficiency of developing, and ensuring the quality of code written.

The code editor we have primarily used is VS Code, this offers code highlighting, debugging capabilities, extensibility and an integrated terminal to improve the development process greatly.

We also used Git to implement version control over the code, this is so that we are able to manage changes to the code and collaborate easily as new features can be safely tested, and if we find errors with the new feature, the original version of the code can be retrieved easily.

Finally, we used Postman to test our APIs. It is very effective for sending HTTP requests and analyzing what the backend services returned, ensuring our API is working as it should do, before implementation to the frontend.

CHAPTER 6

PROJECT DESCRIPTION

This chapter explains the design and implementation of the CampusBuy system in details. It introduces the core modules and functions which were built in this application.

6.1 User registration & authentication

A secure registration and authentication for the system are incorporated so that only authenticated users can log into the system. After registration is done users are then allowed to log in and access all system facilities, this will increase system security as only true campus members will be able to login thus eliminating fake account creation.

6.2 Product listing module

This product listing module provides users a way to create and manage the list of product they want to sale. The users can enter product's title, description, price, category and product image. The system supports following features:

- Each product is attached to a seller account
- Each listing is up-datable or deletable any time
- All the products can be uploaded and managed through cloud system.

The module design should be easy to use by the seller.

6.3 Search and filtering of Product

In order to make the system user-friendly, we developed search and filter functionality. User will have ease of search product using keyword, or category based.

In order to help the search further, we also provide following filter for product:

- Filter by category
- Filter by price range
- Show newly added products

This help user to find any related product in ease.

6.4 Wishlist Functionality

Allows the user to add products they like for the future, the feature enhance convenience of the user and interactivity.

Functionalities:

- Add to wishlist
- Delete from wishlist
- Access to the products in wishlist

It helps the users to manage the products that can be purchased for the future, enhance the user experience.

6.5 Real Time Chat System

The real-time chat system permits buyers and sellers to interact directly with each other. This system makes use of Socket-based technology to deliver instant messages.

Important characteristics of the chat system include:

- Instant message delivery
- Chat history retrieval
- User friendly chat interface

The system allows buyers and sellers to talk about product features, negotiate prices and agree upon purchase details without leaving the platform.

6.6 Complaint/Report module

There is a complaint/report module so that the application can be a safe and honest place. User may report products or other users. The system supports easy submission of complains, status checking and administrative handling of complains. This module helps to implement discipline on the platform.

6.7 Admin dashboard and Analytics

Through the admin dashboard, admin will have the total control over the system. Through this, admin can observe the whole activity and manage the platform.

- Functions of admin dashboard:

- Users control (view/block/delete users)
- Products moderation (delete inappropriate listings)
- Complaint resolution
- Basic analysis (user activities, total listings, etc.)

6.8 Security Features

Security is of a high priority in the implementation of CampusBuy. Several techniques are employed to ensure the security of the platform usage.

Such mechanisms are:

- OTP user verification
- Safe API interaction
- Data validation to prevent misuse of the platform
- User/admin level access

CampusBuy, which covers all the basic modules needed for a campus marketplace has been implemented. Each module ensures that it is user friendly, secure, efficient. And by its modular nature, can be extended to include new functionalities in the future.

CHAPTER 7

TESTING AND VALIDATION

The final stage is testing where you confirm that your system works as it is expected, satisfies the user needs and does not have critical faults. Testing the CampusBuy system ensures that it runs perfectly and can perform, reliability, securely and be usably tested.

The testing phase will be divided into units, integration, and system testing to confirm that each part of the system has been tested and works when combined with other parts.

7.1 Testing Methodologies

While the CampusBuy application was being developed, the following testing methodologies were utilized:

- Black Box Testing - It tests the internal structure of code and focus to the input and output of the application.
- White Box Testing - It tests internal code structure, logic and data flow.
- Manual Testing - The system was tested manually by implementing test cases to test its functions.
- Functional Testing - Verifies each and every function as required by the system.

7.2 Unit Testing

Unit testing is the testing of the individual units of the program to check whether they are working or not.

In CampusBuy we had applied unit testing for:

- Module for user registration and login.
- Module for listing and management of product.
- Module for search and filter.
- Module for handling chat message.

We had applied unit testing with valid and invalid inputs for each module. This had helped us to check the validation and handling of error.

7.3 Integration Testing

Integration testing verifies how different modules of the system interact with each other.

In CampusBuy system, integration testing assured:

- Communication between frontend and backend APIs to work fine
- Data flow between server and database is handled fine
- chat system is integrated with the authentication system fine
- Modules such as product listing and searching interaction to be fine.

7.4 System Testing

System testing refers to the testing of the application as an entire system. This testing ensured that all the functional and non-functional requirements of the application were fulfilled.

The system was tested for:

- System function as a whole
- User Interface behavior
- Performance under normal load
- Security and access controls.

It was ensured that the system testing validated that all the modules interacted with each other seamlessly.

7.5 Test cases and results

Each of the following test cases verify certain system functionality, containing the input data, expected output, and actual output.

Observations:

- All of the primary features (login, product listing, chat, complaint system) worked properly as expected.
- All the error handling mechanisms successfully prevented undesired operations.
- The real time chat worked efficiently without lag time.

Summary:

- Almost all the test cases passed correctly

- A couple of issues with UI and validation were addressed
- No severe system crash happened

This ensures that the system satisfy the needs.

7.6 Bug Tracking and Fixes

During the testing of the system, a number of smaller bugs and issues were encountered. These bugs were then fixed to improve the performance and ease of use of the system.

General bugs identified were:

- Problems with the input validation in forms
- Small visual bugs on the user interface
- Some delays in the messages being updated (initially)
- Some error messages were not quite right

Bugs that were fixed:

- Improvements to form validation and error checking
- Refinements made to the design of the user interface to ensure consistency
- Optimisation of real-time communication through the use of socket
- Error messages were checked and some corrected.

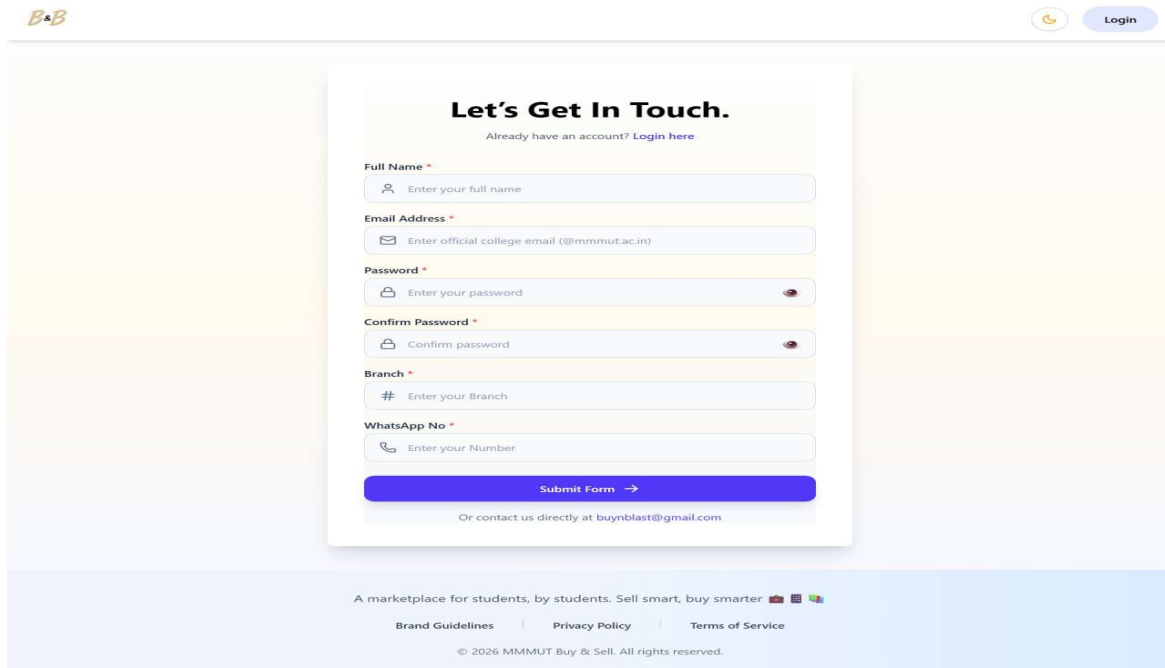
Each issue that arose was tracked and addressed methodically to ensure that it would not cause further problems with the running of the system.

Testing and validation has now verified that CampusBuy system has functionality, stability and security. With variety of testing procedures, the errors have been minimized to an acceptable level and the performance has been enhanced. The application is now ready for implementation.

CHAPTER 8

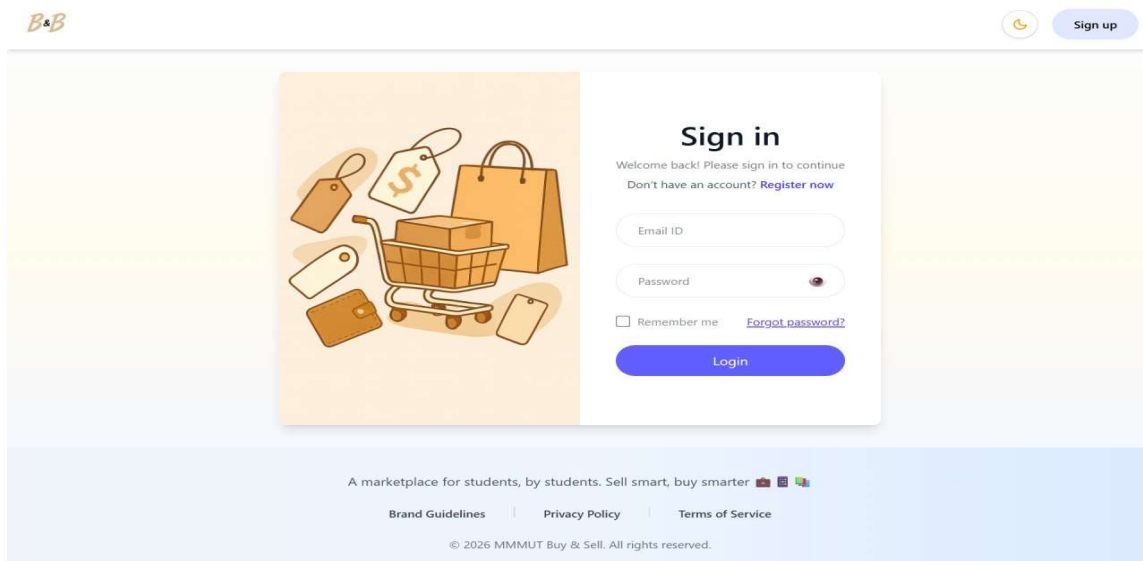
OUTPUT SCREENS

In this chapter, we discuss the result after the implementation and testing the CampusBuy system.



The screenshot shows the registration page of the CampusBuy system. The page has a light yellow background with a white central form. The form is titled "Let's Get In Touch." and includes a link "Already have an account? Login here". The form fields are: Full Name (with a person icon), Email Address (with an envelope icon), Password (with a lock icon), Confirm Password (with a lock icon), Branch (with a hash icon), and WhatsApp No (with a phone icon). Each field has a placeholder text. Below the fields is a blue "Submit Form" button with a right arrow. At the bottom of the form, it says "Or contact us directly at buynblast@gmail.com". The footer of the page includes the text "A marketplace for students, by students. Sell smart, buy smarter" with icons, and links for "Brand Guidelines", "Privacy Policy", and "Terms of Service". The copyright notice is "© 2026 MMMUT Buy & Sell. All rights reserved."

Fig 4. Registration Page



The screenshot shows the login page of the CampusBuy system. The page has a light yellow background with a white central form. The form is titled "Sign in" and includes a welcome message "Welcome back! Please sign in to continue" and a link "Don't have an account? Register now". The form fields are: Email ID and Password (with a lock icon). Below the fields is a "Remember me" checkbox and a link "Forgot password?". At the bottom of the form is a blue "Login" button. To the left of the form is an illustration of shopping bags, a shopping cart, and a smartphone. The footer of the page includes the text "A marketplace for students, by students. Sell smart, buy smarter" with icons, and links for "Brand Guidelines", "Privacy Policy", and "Terms of Service". The copyright notice is "© 2026 MMMUT Buy & Sell. All rights reserved."

Fig 5. Login Page

Fig 6. Update Profile Page

Fig 7. List Product Page

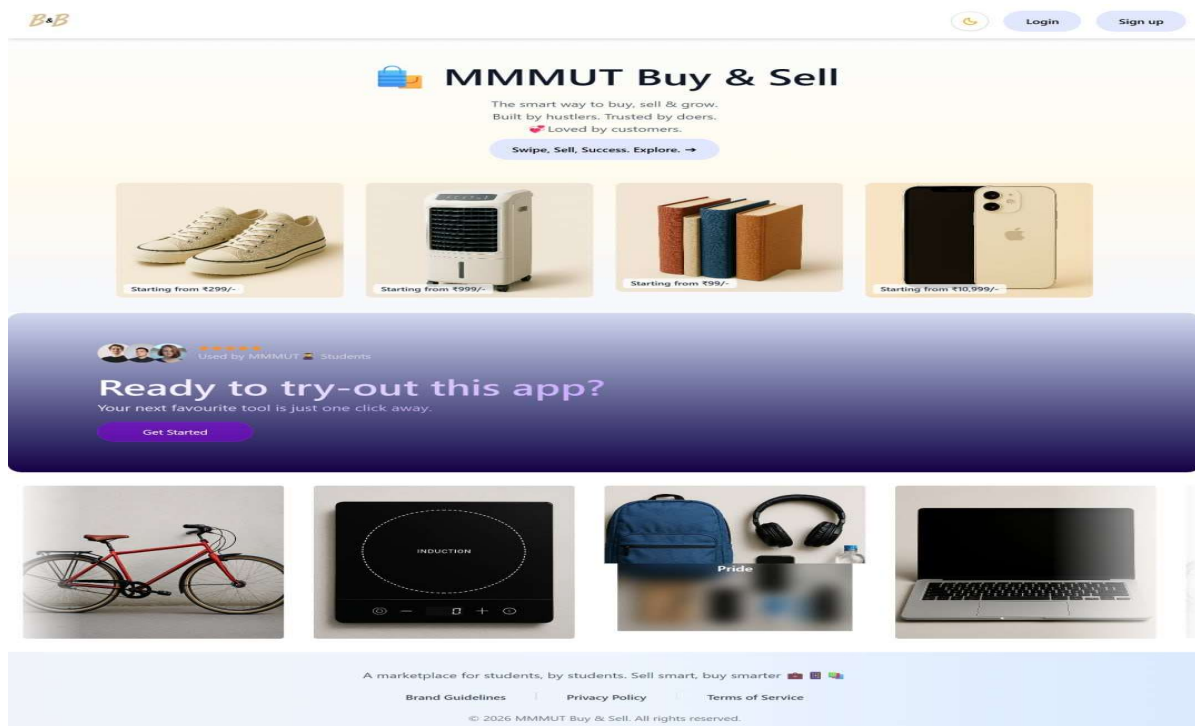


Fig 8. Home Page

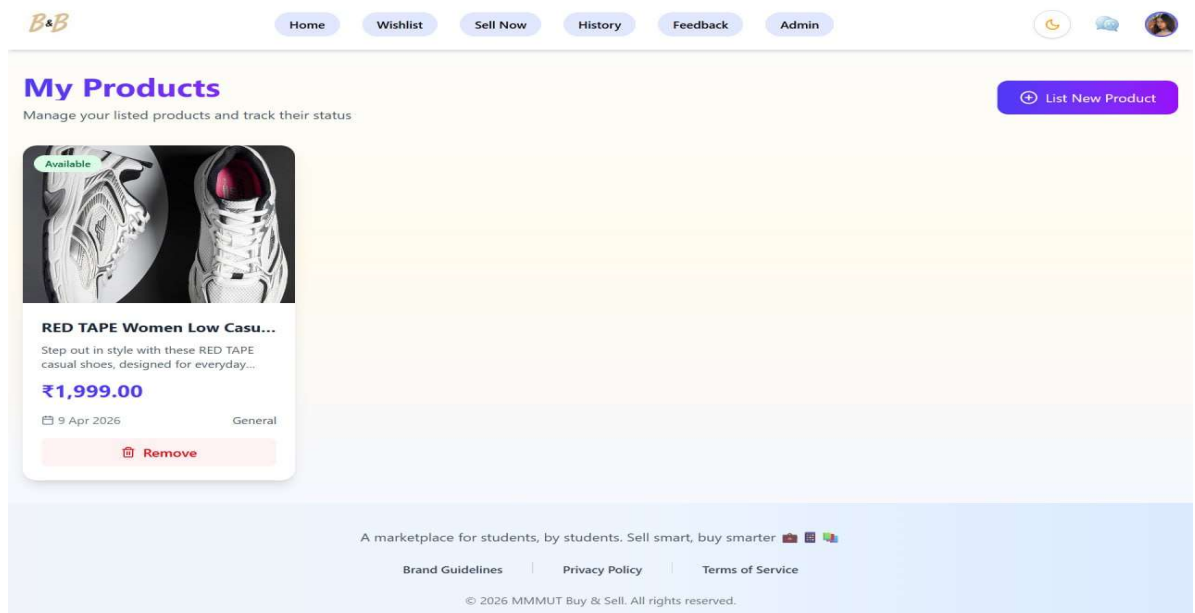


Fig 9. My Products Page

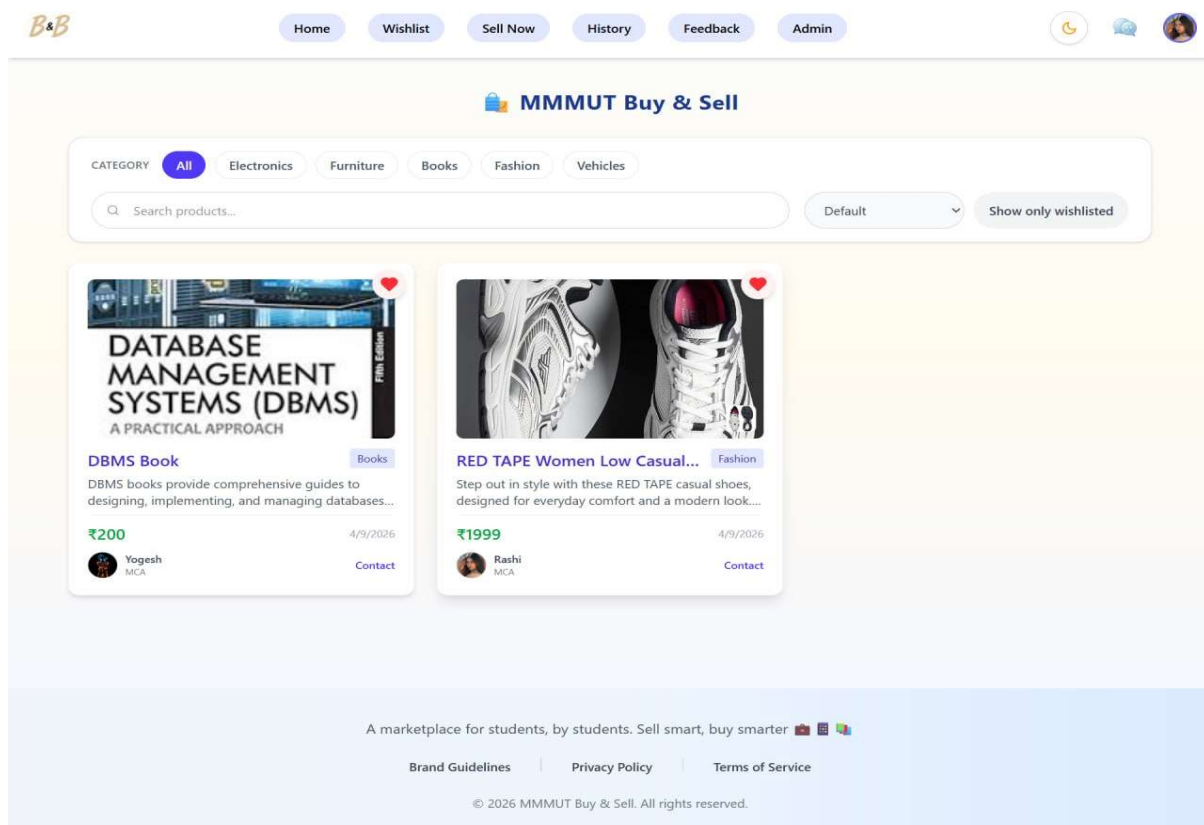


Fig 10. All Products Page

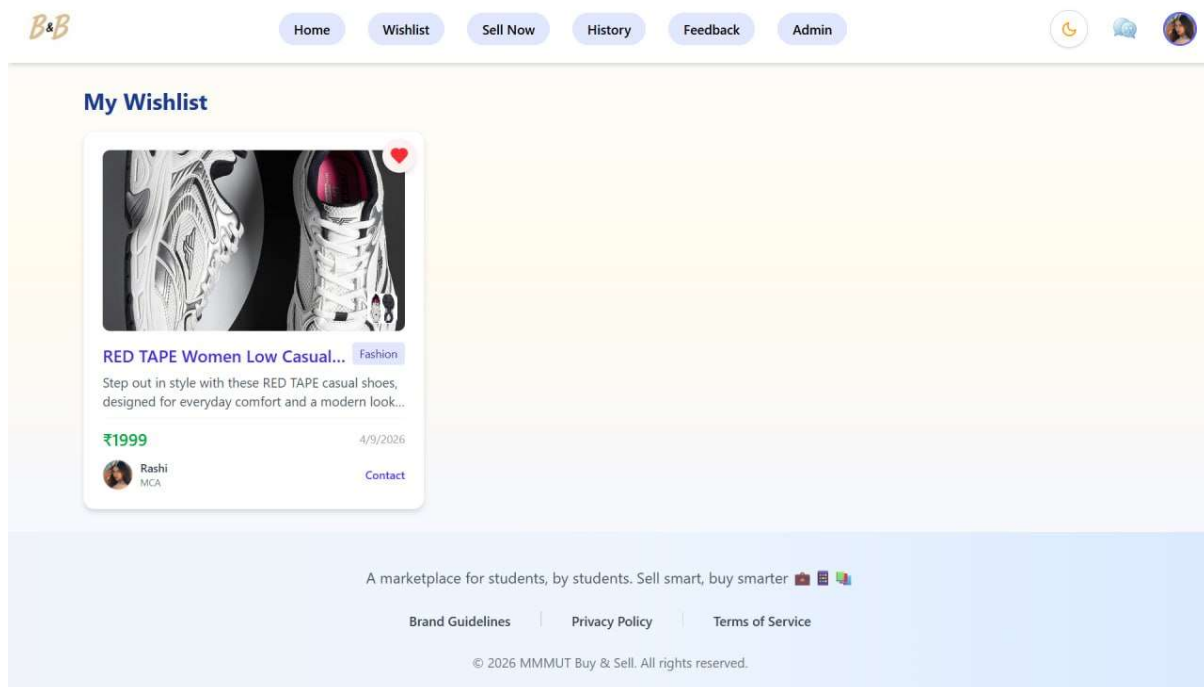
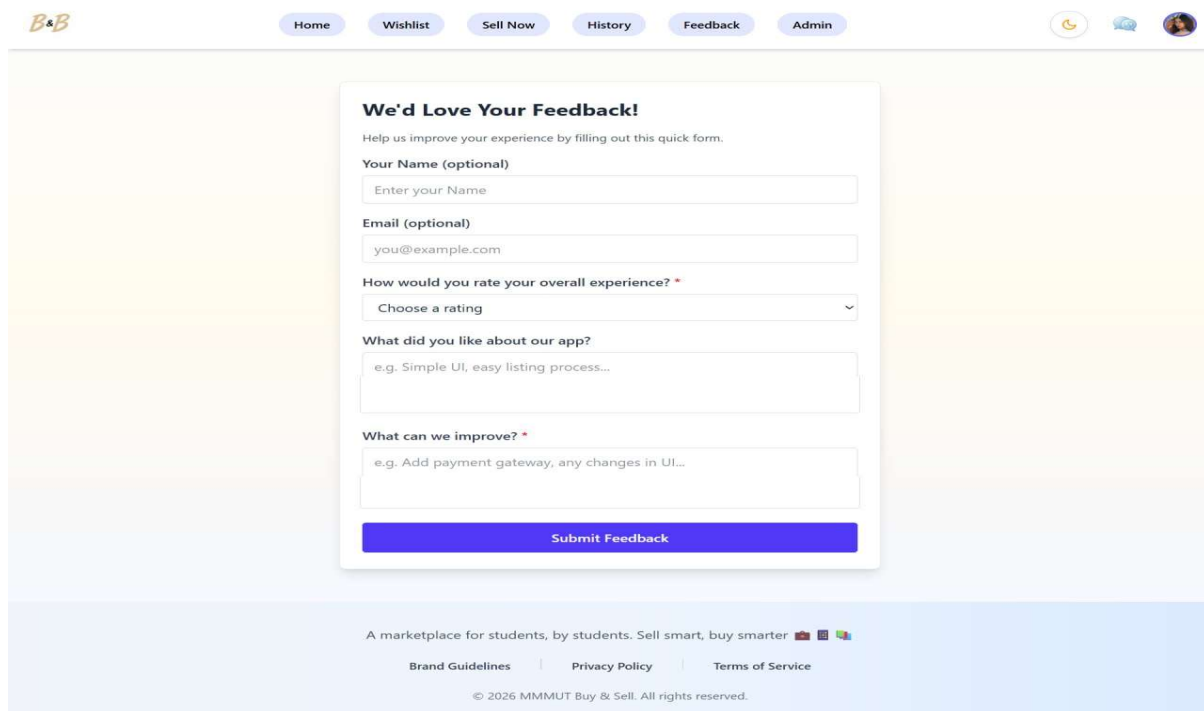
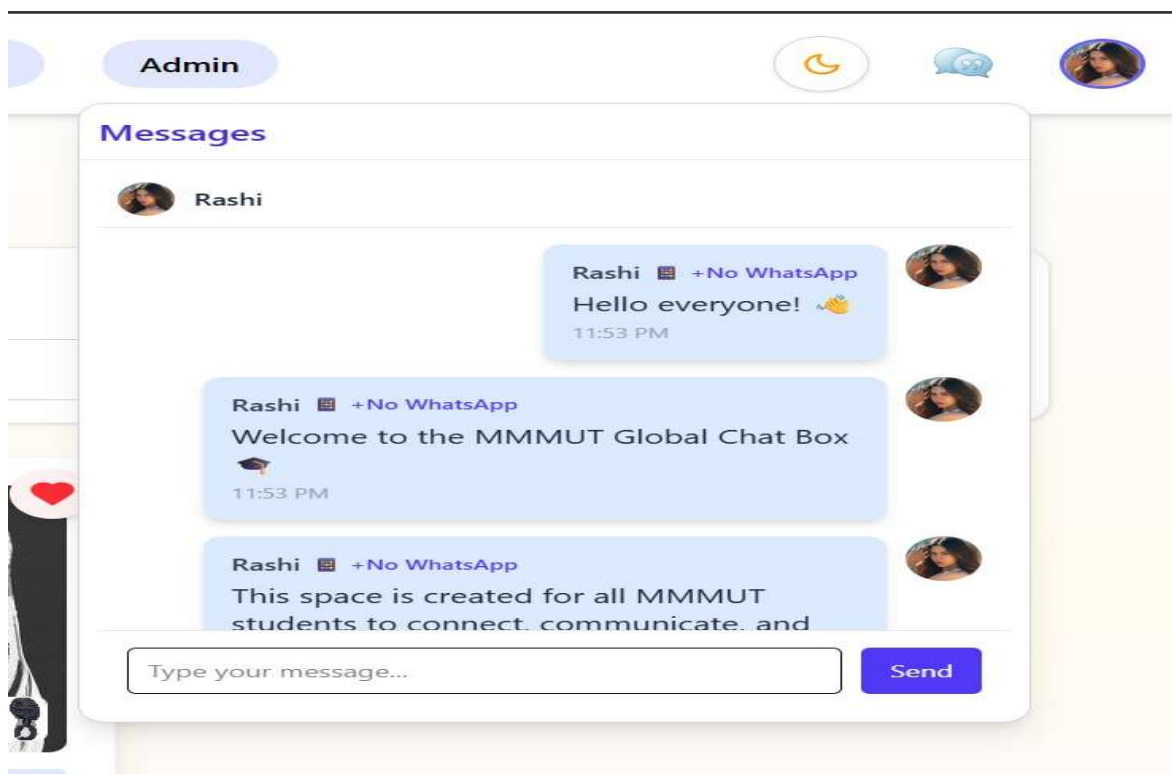


Fig 11. Wishlist Page



The screenshot shows a feedback form titled "We'd Love Your Feedback!". The form includes fields for "Your Name (optional)", "Email (optional)", and a rating dropdown. It also has two text areas for "What did you like about our app?" and "What can we improve?". A "Submit Feedback" button is at the bottom. The page has a navigation bar with "Home", "Wishlist", "Sell Now", "History", "Feedback", and "Admin". The footer contains the text "A marketplace for students, by students. Sell smart, buy smarter" and copyright information for MMMUT Buy & Sell.

Fig 12. Feedback Page



The screenshot shows a chat interface titled "Messages" with a contact named "Rashi". The chat history includes three messages from "Rashi" with a status of "+No WhatsApp". The messages are: "Hello everyone!", "Welcome to the MMMUT Global Chat Box", and "This space is created for all MMMUT students to connect, communicate, and". At the bottom, there is a text input field labeled "Type your message..." and a "Send" button.

Fig 13. Universal Chat Page



[Home](#)
[Wishlist](#)
[Sell Now](#)
[History](#)
[Feedback](#)
[Admin](#)





Admin Dashboard

Total Users
6

Total Products
2

[Users](#)
[Products](#)

Name	Email	Role	Action
Rashi	2024073045@mmmut.ac.in	admin	Remove User
Richa	2024073047@mmmut.ac.in	user	Remove User
Mridushi	2024073029@mmmut.ac.in	user	Remove User
preeti	2024073038@mmmut.ac.in	user	Remove User
Priyanshu	2024073039@mmmut.ac.in	user	Remove User
Yogesh	2024073074@mmmut.ac.in	user	Remove User

A marketplace for students, by students. Sell smart, buy smarter 🛒📱💻

[Brand Guidelines](#)
[Privacy Policy](#)
[Terms of Service](#)

© 2026 MMMUT Buy & Sell. All rights reserved.

Fig 14. Admin Dashboard Page

CHAPTER 9

CONCLUSION AND FUTURE WORK

9.1 Conclusion

The project known as CampusBuy was created to make buying and selling of goods on the campus secure and organized. Prior traditional methods of selling and buying goods using campus notice boards and social media groups, was slow, disorganized and did not involve a sense of trust between users. CampusBuy had successfully overcome such limitations and provided an organized and user-friendly system just for the college students.

Features included in the application are, the authentication of users, listing of products to buy, real time chat feature between the sellers and buyers, a wishlist option, and also a reporting mechanism. The application provides a streamlined experience of buying and selling things on the campus. Modern web technologies such as the MERN stack were utilized which not only helped to make the system efficient and reliable but also scalable. Security was maintained on the platform by providing access only to registered students and also implementing admin privileges and a reporting tool.

9.2 Future Work

In spite of offering a fully functional platform currently, CampusBuy could be improved by implementing the following future enhancements:

- Online Payment Integration: Integrating online payment gateways would enable users to make transactions right inside the app, which in turn is very important in the context of transactions made between two students over a wide campus;
- Mobile application: A mobile application would further enhance accessibility and usage among students;
- Enhanced Recommendation system: AI based recommendations could make users more convenient in searching for suitable products;
- Rating and Review system: A rating and review system for students could enable greater transparency among users;
- Location based services: GPS positioning within the campus would facilitate smooth exchanges and transactions;

- Notification system: Alerting students to any message, new updates on products or promotions through real-time notifications;
- Multi-campus support: With proper scaling up, this service could serve to different campuses while maintaining individual ecosystems.

CHAPTER 10

REFERENCES

- **Facebook Inc., React** - A JavaScript library for building user interfaces. Available at: <https://reactjs.org/>
- **Node.js Foundation, Node.js Documentation.** Available at: <https://nodejs.org/>
- **Express.js**, Fast, unopinionated, minimalist web framework for Node.js. Available at: <https://expressjs.com/>
- **MongoDB Inc., MongoDB Documentation.** Available at: <https://www.mongodb.com/docs/>
- **Socket.IO**, Real-time bidirectional event-based communication. Available at: <https://socket.io/docs/>
- **Cloudinary**, Cloud-based image and video management. Available at: <https://cloudinary.com/documentation>
- **Tailwind Labs, Tailwind CSS Documentation.** Available at: <https://tailwindcss.com/docs>
- **Mozilla Developer Network (MDN), Web Development Resources.** Available at: <https://developer.mozilla.org/>
- **JWT.io**, Introduction to JSON Web Tokens. Available at: <https://jwt.io/introduction>
- **GitHub Inc., Version Control and Collaboration Platform.** Available at: <https://github.com/>
- **Postman Inc., API Development and Testing Tool.** Available at: <https://www.postman.com/>